FPT UNIVERSITY MSS301 - Assignment 01 **MSS301 - Assignment 01**

Time allowed: 90 minutes. Submit on Classroom before the deadline.

Total points: 100

1 Instructions

- Build a backend system using **Spring Boot** with the **microservices architecture** and **Spring Cloud Gateway** shown below. Follow the N-layer architecture (Controller – Service Interface + Implementation – Repository) taught in the course.
- **PLAGIARISM**: Copying from your classmate(s) or AI-based tools will result in **ZERO** points.
- Technologies: JDK 21 or later, Spring Boot 3.x, Spring Cloud (Gateway + OpenFeign), SQL Server 2019+ (Spring Data JPA). Maven or Gradle.
- Follow the **database-per-service** principle: each microservice connects to its own database. There is no shared database and no cross-service foreign key.
- Clients are only allowed to call the system through the API Gateway at http://localhost:8080. All the JSON responses below are references – field names must match **exactly**.
- Use the provided seed script (seed.sql) to initialize the two databases before testing.

2 Context

FUCarRentingSystem rents cars to its customers. In this exam you build **two microservices** and **one API Gateway**:

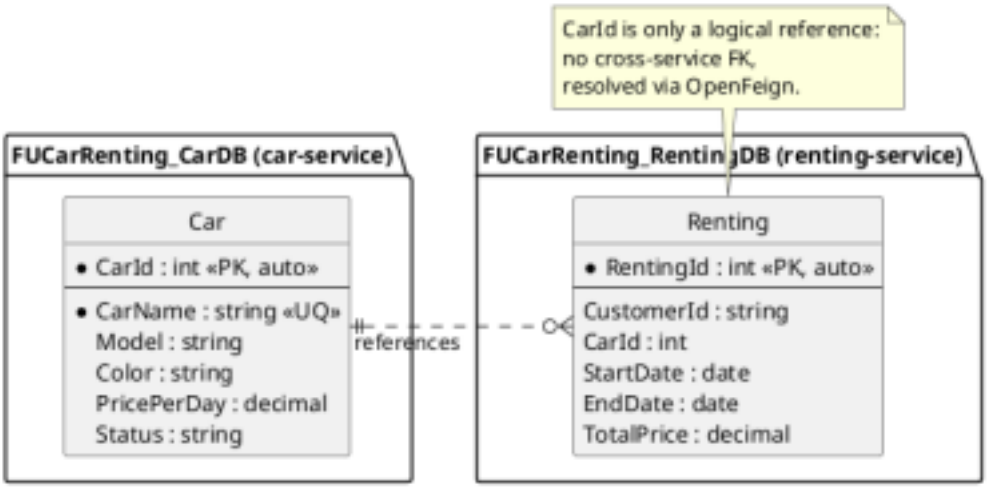
- **car-service** (port 8081): manages the car fleet (CRUD + status).
- **renting-service** (port 8082): manages renting transactions. When a new renting request arrives, it must call **car-service via OpenFeign** to load the car and verify it is available.
- **api-gateway** (port 8080, Spring Cloud Gateway): the single entry point for all clients; routes incoming requests to the correct internal service.

Client → API Gateway :8080 → car-service :8081 / renting-service :8082

renting-service —OpenFeign——→ car-service

Database design (SQL Server, **database-per-service** – each service owns its own database): • FUCarRenting_CarDB (owned by car-service): table Car.

- FUCarRenting_RentingDB (owned by renting-service): table Renting.
- There is **no cross-service foreign key**: Renting.CarId only references Car.CarId logically and is resolved by renting-service through OpenFeign at runtime.



3 API Specification

All endpoints are exposed **through the gateway** on port 8080. Responses are JSON.

3.1 car-service

ID	Method	URL mapping	Description
API-01	POST	/api/cars	Create a car
API-02	GET	/api/cars	List all cars
API-03	GET	/api/cars/{id}	Get one car by id
API-04	PUT	/api/cars/{id}	Update a car
API-05	DELETE	/api/cars/{id}	Delete a car

3.2 renting-service

ID	Method	URL mapping	Description
API-06	POST	/api/rentings	Create a renting transaction
API-07	GET	/api/rentings	List all renting transactions

Request input / Response output (JSON)

API-01 – POST /api/cars – input:

```
{
   "carName": "VinFast VF8",
   "model": "VF8 Plus",
   "color": "Red",
   "pricePerDay": 1200000,
   "status": "Available"
}
```

Output – HTTP 201 Created with the stored car:

```
{
   "carId": 6,
   "carName": "VinFast VF8",
   "model": "VF8 Plus",
   "color": "Red",
   "pricePerDay": 1200000,
   "status": "Available"
}
```


API-02 – GET /api/cars: returns HTTP 200 with an array of car objects (same shape as above). **API-03 – GET** /api/cars/{id}: returns HTTP 200 with one car; if the id does not exist it returns 404 with body {"error": "Car not found"}.

API-04 – PUT /api/cars/{id}: same input body as API-01 (all fields required). Output: 200 with the updated car; 404 if the id does not exist.

API-05 – DELETE /api/cars/{id}: returns 204 No Content on success. If the car status is Rented, there is **no** delete in database, and it returns 409 Conflict with body {"error": "Car is rented"}. **API-06 – POST** /api/rentings – input:

```
{
   "customerId": "CUST-001",
   "carId": 1,
   "rentStartDate": "2026-10-15",
   "rentEndDate": "2026-10-17"
}
```

Business logic: the renting-service calls car-service (OpenFeign) to load the car, verifies the car exists and its status is Available, computes TotalPrice = PricePerDay * number of renting days, stores the transaction, and sets the car status to Rented.

Output – HTTP 201 Created with the transaction (car info must come from the Feign call):

```
{
   "rentingId": 3,
   "customerId": "CUST-001",
   "carId": 1,
   "carName": "Toyota Camry",
   "carModel": "Camry 2.5Q",
   "rentStartDate": "2026-10-15",
   "rentEndDate": "2026-10-17",
   "totalPrice": 3000000
}
```

Error cases (no transaction is stored):

- Any required field missing/blank → 400 with {"error": "Invalid data"}
- Car id does not exist → 404 with {"error": "Car not found"}
- Car status is Rented → 409 with {"error": "Car is rented"}
- rentEndDate ≤ rentStartDate → 400 with {"error": "Invalid data"}

API-07 – GET /api/rentings: returns 200 with an array of renting transactions (same shape as the API-06 output). Because Renting only stores CarId, the renting-service must call car-service (**OpenFeign**) for each transaction to fill in carName and carModel.

4 Tasks

4.1 car-service (port 8081)

1. **Implement** POST /api/cars. Validate: all fields required and pricePerDay > 0; on invalid input [15 points] return 400 with {"error": "Invalid data"}.

2. **Implement** GET /api/cars returning all cars. [5 points] 3. **Implement** GET /api/cars/{id} as specified (200 / 404). [10 points] 4. **Implement** PUT /api/cars/{id} as specified (200 / 400 / 404). [10 points] 5. **Implement** DELETE /api/cars/{id} as specified (204 / 409 when the car is Rented). [10 points]

4.2 renting-service (port 8082)

1. **Implement** POST /api/rentings happy path: call car-service via **OpenFeign**, verify the car is [20 points] Available, compute totalPrice, persist the transaction, set the car status to Rented, and return 201 with the JSON above (car info from the Feign response).

2. **Handle** the error cases of the creating-renting task (400 / 404 / 409) with the specified bodies instead [10 points] of an uncaught exception.

3. **Implement** GET /api/rentings returning all transactions. [5 points]

4.3 API Gateway (port 8080)

1. **Configure** Gateway routing so that all /api/cars/** requests reach car-service and /api/rentings/** [8 points] requests reach renting-service. Clients never call internal ports directly.

2. **Verify and document**: the two routes must behave transparently – request bodies, headers, and [7 points] JSON responses pass through unchanged. Any other path returns 404.

— THE END —